DATA STRUCTURES & ALGORITHMS PROJECT

AutoComplete Search Engine

using Trie Data Structure

A Full-Stack Implementation with C++ Backend & Web Frontend

Submitted By

Shivom Singh
Aditya Prakash
Shelar Yash Kanifnath
Tumma Abhinav

Academic Year 2024–2025

Abstract

This project presents the design and implementation of an AutoComplete Search Engine built upon the Trie (Prefix Tree) data structure. The system provides real-time word suggestions, exact-match search with frequency tracking, and fuzzy spell correction using dynamic programming-based edit distance computation.

The backend is engineered in C++ using a single-header HTTP server library (cpp-httplib), exposing a RESTful API on port 8080. The frontend is a modern, responsive web application styled with glassmorphism aesthetics, communicating with the backend over HTTP. Together, the system demonstrates how classical data structures can power practical, production-grade software features.

The system loads a corpus of 5,000+ common English words and supports live insertions, making it dynamic and extensible at runtime.

1. Introduction

1.1 Motivation

Autocomplete is one of the most ubiquitous features in modern software — from Google Search to IDE code editors. Behind every such feature lies an efficient data structure capable of prefix-based retrieval. This project was conceived to demonstrate that a Trie, one of the most elegant tree-based structures in computer science, can serve as the backbone of a fully functional, real-world search engine.

1.2 Problem Statement

Given a large dictionary of English words, build a system that can:

- Instantly suggest relevant completions as a user types a prefix.
- Track the popularity (frequency) of searched words.
- Return fuzzy corrections for misspelled queries.
- Allow dynamic insertion of new words at runtime.
- Serve all results over an HTTP API accessible from any browser.

1.3 Scope

The project covers the full software stack — data structure design, algorithm implementation, API development, and frontend engineering — providing a comprehensive demonstration of how DSA concepts translate into real-world applications.

2. Data Structure: The Trie

2.1 What is a Trie?

A Trie (also known as a Prefix Tree or Digital Tree) is a multi-way tree data structure used for storing strings over an alphabet. Unlike binary search trees, Tries store characters at each node, allowing efficient prefix-based lookups. Each path from the root to a marked node represents a word in the dictionary.

Key properties of a Trie:

- Every node has up to 26 children (one per lowercase letter of the alphabet).
- The root node represents an empty prefix.
- Each node stores an isEnd flag and a frequency counter.
- All descendants of a node share the same prefix.

2.2 TrieNode Structure

Each node in the Trie is an instance of the TrieNode class, defined as follows:

```
class TrieNode {
public:
    TrieNode* children[26];  // Pointers to child nodes (a-z)
    bool      isEnd;         // True if this node ends a valid word
    int       frequency;     // Search/usage frequency counter
};
```

2.3 Time & Space Complexity

Operation	Time Complexity	Space Complexity
Insert Word	O(L)	O(L × 26)
Search Word	O(L)	O(1) extra
Prefix Suggestions	O(L + N)	O(N) output
Top-K Suggestions	O(L + N log N)	O(N)
Fuzzy Correction	O(W × L^2)	O(L^2) per word

Where L = length of the word/prefix, N = number of matching words, W = total words in dictionary.

3. System Architecture

3.1 Overview

The system follows a classic client-server architecture. The C++ backend manages the Trie data structure and serves an HTTP API. The HTML/CSS/JavaScript frontend runs in the browser and communicates with the backend via Fetch API calls.

```
[ Browser / Frontend ] ←── HTTP (port 8080) ──→ [ C++ Server / Trie
                                                Engine ]
```

3.2 Backend (server.cpp)

The backend is a self-contained C++ application that:

- Loads the 5,000-word corpus from 5000-more-common.txt at startup.
- Constructs the Trie in memory (O(N × L) time).
- Listens on 0.0.0.0:8080 for incoming HTTP requests.
- Handles CORS headers for cross-origin browser requests.
- Serializes all responses as JSON without any third-party library.

3.3 Frontend (index.html)

The frontend is a single-page application implemented in vanilla HTML, CSS, and JavaScript. Its features include:

- Real-time prefix autocomplete triggered on every keystroke.
- A modal-based control panel for all five operations.
- Frequency badges displayed alongside each suggestion.
- Glassmorphism UI design with a purple-gradient dark theme.
- Responsive two-column layout adapting to screen width.

3.4 File Structure

File	Description
`server.cpp`	Main C++ source — Trie class + HTTP routes
`httplib.h`	Single-header HTTP library (cpp-httplib)
`server.exe`	Compiled Windows executable
`index.html`	Frontend SPA — UI, API calls, and styling
`5000-more-common.txt`	Word corpus — one word per line

4. API Reference

The backend exposes four HTTP endpoints on localhost:8080. All responses are Content-Type: application/json and include CORS headers.

4.1 GET /suggest?prefix=<text>

Returns the top-5 word suggestions ranked by frequency for the given prefix.

```
Request:   GET /suggest?prefix=hel
Response:
{"prefix":"hel","suggestions":[{"word":"hello","frequency":3},{"word":"hel
p","frequency":1}]}
```

4.2 GET /search?word=<text>

Performs an exact dictionary lookup and increments the word's frequency on each hit.

```
Request:   GET /search?word=hello
Response:  {"word":"hello","found":true,"frequency":4}
```

4.3 GET /correct?word=<text>

Returns up to 5 fuzzy corrections for potentially misspelled words using edit distance ≤ 2.

```
Request:   GET /correct?word=helo
Response:  {"typo":"helo","corrections":[{"word":"hello","frequency":3}]}
```

4.4 POST /insert

Inserts a new word (with optional frequency) into the Trie at runtime.

```
Request:   POST /insert
Body:      {"word":"quantum","frequency":5}
Response:  {"success":true,"word":"quantum","frequency":5,"message":"Word
inserted successfully"}
```

5. Algorithm Details

5.1 Insertion Algorithm

Word insertion traverses the Trie character by character, creating nodes as needed, and sets the isEnd flag and increments frequency at the terminal node. The word is normalized to lowercase before processing.

5.2 Prefix-Based Autocomplete

The getTopK function first navigates to the node corresponding to the last character of the prefix, then performs a DFS (collectAllWords) to gather all words reachable from that node. Results are sorted by frequency in descending order and truncated to K results.

5.3 Fuzzy Spell Correction

The fuzzyCorrect method computes the Levenshtein edit distance between the query and every word in the dictionary. Only words with 0 < distance ≤ 2 are included, ensuring the query itself (distance 0) is excluded. The algorithm uses dynamic programming:

Recurrence relation:

```
dp[i][j] = dp[i-1][j-1]                          if a[i-1] == b[j-1]
         = 1 + min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])  otherwise
```

This achieves $O(L^2)$ time and space per word pair, where L is the length of the longer word.

5.4 Frequency-Aware Ranking

Every word carries an integer frequency counter. Insert operations add to the frequency; search operations increment it by one. Suggestions and corrections are always sorted by frequency descending, ensuring the most popular words appear first.

6. Team Contributions

The project was collaboratively designed and implemented by a team of four. Below is an overview of each member's primary contributions.

Team Member	Role	Key Contributions
Shivom Singh	*Backend Lead*	Trie class design, insert/search/frequency logic, JSON serialization, HTTP server setup
Aditya Prakash	*Algorithm Specialist*	Edit distance DP implementation, fuzzy correction engine, Top-K sorting algorithm
Shelar Yash Kanifnath	*Frontend Engineer*	UI/UX design, HTML/CSS glassmorphism layout, modal system, real-time autocomplete
Tumma Abhinav	*Integration & QA*	API integration, CORS configuration, corpus loading, end-to-end testing & documentation

7. Results & Testing

7.1 Performance Observations

The system was tested on a Windows machine with the provided 5,000-word corpus. All operations performed within acceptable time bounds:

- Startup corpus loading: < 50 ms for 5,000 words.
- Autocomplete response: < 5 ms for any prefix query.
- Exact search: O(L) — essentially instantaneous.
- Fuzzy correction: ~100–300 ms for a 5,000-word dictionary (acceptable for one-time calls).

7.2 Sample Test Cases

Endpoint	Input	Expected Output
`/suggest`	`prefix=pro`	produce, problem, process, program, project
`/search`	`word=hello`	found: true, frequency: 1+
`/correct`	`word=recieve`	receive (edit distance 2)
`/insert`	`word=trie, freq=10`	success: true, word inserted
`/suggest`	`prefix=xyz`	Empty array (no matches)

8. Conclusion & Future Scope

8.1 Conclusion

This project successfully demonstrates the practical power of the Trie data structure in a real-world application. By integrating a high-performance C++ Trie engine with a REST API and a modern web frontend, the team has produced a fully functional AutoComplete Search Engine that rivals the core functionality found in production search systems.

Key achievements include:

- A complete, hand-rolled Trie implementation in C++ with no external data structure libraries.
- A clean, well-documented REST API with CORS support.
- A polished, responsive frontend with real-time autocomplete.
- Fuzzy spell correction implemented from first principles using dynamic programming.

8.2 Future Scope

Several enhancements could significantly extend this project:

- Persistent Storage: Serialize the Trie to disk (JSON/binary) so inserted words survive server restarts.
- Weighted Corpus: Load words with pre-assigned frequency weights from a real search log.
- Delete Operation: Implement word deletion with proper Trie pruning.
- Ranked Fuzzy Matching: Weight corrections by both frequency and edit distance.
- Multi-language Support: Extend the alphabet size beyond 26 to support Unicode characters.
- HTTPS & Authentication: Secure the API for deployment in production environments.

References

1. Knuth, D. E. (1997). The Art of Computer Programming, Vol. 3: Sorting and Searching. Addison-Wesley.

2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

3. Hirose, Y. cpp-httplib: A C++ header-only HTTP/HTTPS server and client library. https://github.com/yhirose/cpp-httplib

4. Levenshtein, V. I. (1966). Binary Codes Capable of Correcting Deletions, Insertions, and Reversals. Soviet Physics Doklady, 10(8), 707–710.